# Programming Assignment: Search

**Deadline**
To remain eligible for grade **A** in LAB2, both the required Kattis submission and the individual Assignment Learning Journal must be submitted no later than **23 November 2026 at 19:00**. A late submission of either component results in a maximum LAB2 grade of **B**.

**Individual and pair work**
You are encouraged to develop the program in pairs, but you may also work individually. Groups larger than two are not permitted.
Even when the program is developed in a pair, each student must submit an **individual Assignment Learning Journal**. You will be assessed individually and must be able to explain the relevant theory, your design decisions, your contribution, and the submitted implementation. Partners may refer to the same implementation and experiments, but each journal must be written individually and reflect the student's own understanding.

**What you submit and how it is assessed**
This assignment has two required submissions:

1. the program submitted to Kattis; and
2. an individual Assignment Learning Journal submitted in Canvas.

The Kattis result determines the highest grade range for which the implementation is eligible. The journal provides evidence of your technical understanding and is used to determine the grade within that range. A TA reviews the journal and code and assigns the final assignment grade. Your journal and code may later be used to prepare questions for your individual oral examination.

# 1 THE KTH FISHING DERBY

In this assignment we will take a look at the KTH fishing derby game, inspired by the famous fishing derby arcade game.
The players in this turn-based game are two fishing boats out at sea (see Figure 1.1). Imagine that you are in command of the green boat (on the left) while the red one (on the right) is the opponent. The boats can move horizontally and have a fishing line attached. There is a hook at the end of the fishing line which can be moved vertically. Consequently, the possible actions for each player are LEFT, RIGHT, UP, DOWN and STAY. Note that we consider a $20 \times 20$ 2D scenario, and the boats cannot cross each other. If the boats are next to each other, for instance with the green at the left position from the red one, and the green boat applies action RIGHT, the action will just result in the same behavior as action STAY. However, the world is round, and action LEFT applied at the leftmost position of the screen means that the boat will appear on the rightmost position (unless the other boat occupies that position).
Each player has to catch as many valuable fish as possible. A fish is caught once its position coincides with the hook's position. Some fish are more valuable than others resulting in different score points per fish. A penalty (negative points) is given for types of fish that should not be caught. The correspondence between the type of fish and number of points is known at the beginning of the game. A player wins if he or she has obtained a higher number of points than the opponent. The game ends either if no fish is left or the game time has passed (the fishing day is over).

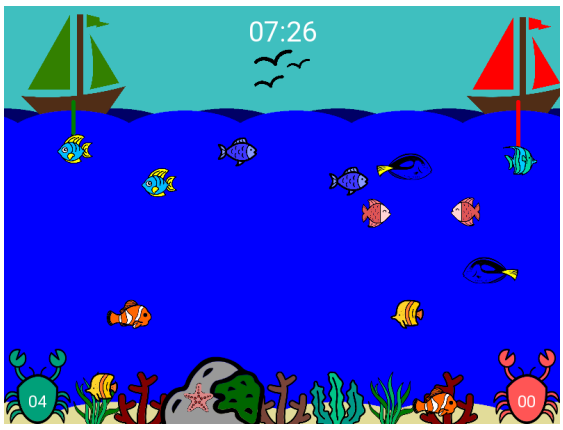



Figure 1.1: Graphical interface of the Fishing Derby game (Minimax). The crabs in the corner show the current score. 07:26 is the time remaining till the end of the fishing day (7 minutes and 26 seconds).

Throughout this document, we will use the KTH fishing derby game as an example of a two-player zero-sum game. Your task in this assignment will be to develop an AI agent that decides which move the green boat should make using a variant of the *minimax* algorithm.

## 2 INTRODUCTION TO GAME THEORY

Game theory studies systems where two or more agents try to maximize their own gain by operating in a shared environment. In this assignment, we are interested in a special class of such problems, namely one where two players are in direct conflict (which are commonly called zero-sum games).
To put it formally, a game consists of the following parts:

> $P = \{A, B\}$ is the set of players (which will always be the same in our case because we will only analyze two-player games).
>
> $S$ is the set of possible game states.
>
> $s_0$ is the initial state, the game *always starts* in the initial state.
>
> $\mu : P \times S \to \mathscr{P}(S)$ (where $\mathscr{P}(S)$ is the set of all subsets of $S$, that is, any possible set of game states belongs to $\mathscr{P}(S)$) is a function that given a player and a game state returns the possible game states that the player may achieve with one *legal* move by the current player.
>
> $\gamma : P \times S \to \mathbb{R}$ is a *utility function* that given a player and a state says how "useful" the state is for the player.

A two-player game is called a zero-sum game if $\gamma(A, s) + \gamma(B, s) = 0$ or equivalently $\gamma(A, s) = -\gamma(B, s)$. Informally, each component of the specification has a role in structuring the game-play. More specifically, we have:

> $\mu$ is the part of the specification that dictates the *rules* of the game (by dictating which are the legal moves). Notice that not all states in $S$ need to be reachable by a legal sequence of moves.
>
> $\gamma$ is the part of the specification that drives the game forward, and it is not unique (any positive rescaling of $\gamma$ is also a utility function).

We say that the game is over when it reaches a state $s_t$ such that $\mu(p, s_t) = \emptyset$ and player $p$ is about to play. In this case, $s_t$ is called a *terminal state*, and we say that:

> Player $A$ wins if $\gamma(A, s) > \gamma(B, s)$.
>
> Player $B$ wins if $\gamma(B, s) > \gamma(A, s)$.
>
> The game is tied if $\gamma(A, s) = \gamma(B, s) = 0$.

Hence, the objective of player $A$ is to reach a terminal state $s_t$ that maximizes $\gamma(A, s_t)$ and, since $\gamma(B, s) = -\gamma(A, s)$, the objective of player $B$ is to reach a terminal state $s'_t$ that minimizes $\gamma(A, s'_t)$.

Questions marked Q1–Q6 are assignment questions. Answer all six in Section J2 of your individual Assignment Learning Journal. Your answers should be concise, use your own words, and connect the concepts to the submitted implementation whenever requested.

**Q1.** Describe the state space, what specifies the initial state of a scenario, and the successor function of the KTH fishing derby game.

**Q2.** Describe the terminal states of the KTH fishing derby game. What utility should be assigned to a terminal state from the perspective of the green/MAX player? If your implementation uses a different terminal utility, briefly justify your choice.

### 2.1 Heuristic functions

For a terminal state, the utility function $\gamma$ gives the exact outcome of the game. In practice, however, minimax cannot normally reach a terminal state within the available time. When the search is cut off at a non-terminal state, an *evaluation function* is used to estimate the utility that would result from continued play.
A simple evaluation function for the KTH fishing derby is:

$$v(A, s) = \text{Score(Green boat)} - \text{Score(Red boat)}$$

This function can be computed efficiently using information contained in the game state.

**Q3.** Why is $v(A, s) = \text{Score(Green boat)} - \text{Score(Red boat)}$ a reasonable evaluation function for a cut-off state? Consider both its relationship to the expected outcome of the game and how efficiently it can be computed.

**Q4.** At what point in the game would $v$ generally provide the most reliable estimate of the final outcome, and why?

**Q5.** Give a concrete example of a state $s$ for which $v(A, s) > 0$, but player B can make a move that causes A to lose. Explain why the evaluation of $s$ is misleading. Consider the simple score-difference evaluation function defined above, even if your implementation uses a different heuristic.

## 2.2 GAME TREES AND MORE HEURISTICS

From the questions in the previous section (specifically the last one) it should be evident that a naive heuristic function is, in general not enough to guarantee a victory against a smart (and sometimes, even random) player. This shortcoming is shared by most evaluation functions. In this section we introduce the minimax algorithm that, given a naive heuristic function, produces a heuristic function that better fits a utility function. To this end we start by introducing the following heuristic function:

$$\eta(A,s) = |w(s,A)| - |l(s,A)| \qquad \eta(B,s) = |w(s,B)| - |l(s,B)|$$

Where $w(s,p)$ is the set of terminal states at which player $p$ wins and which are reachable from state $s$, and $l(s,p)$ is the set of terminal states at which player $p$ loses and which are reachable from state $s$ (and $|\cdot|$ denotes the cardinality of the set, that is, the number of such states).

It is easy to see that $\eta$ satisfies $\eta(A,s) + \eta(B,s) = 0$, and for a terminal state $s_t$ we have that $\eta(A,s_t) > 0$ if player $A$ wins, $\eta(A,s_t) < 0$ if player $B$ wins, and $\eta(A,s_t) = 0$ if it is a draw. Also, note that $\eta$ is not an evaluation function, as its computation requires one to filter through valid sequences of moves.

Now, for simplicity, assume that player $A$ always starts the game. In order to calculate $\eta(A,s)$ it is convenient to represent the game as a *game tree* $GT = (V,E)$ with a set of nodes $V$, each of which corresponds to a game state[1] and a set of edges $E \subseteq V \times V$ satisfying: *Given two nodes from the game tree $s, s' \in V$, the edge $(s,s')$ (also denoted $s \to s'$) is in $E$ if and only if $s'$ can be reached from $s$ with a legal move, i.e., $s' \in \mu(A,s)$ and depth$(s)$ is even, or $s' \in \mu(B,s)$ and depth$(s)$ is odd (see illustration in Figure 2.1).*

A game tree can be used to search for a winning strategy at each point of the game. Particularly, the *complete game tree* is a game tree with the initial state $s_0$ as root and where every possible move is expanded until it reaches a terminal state. We can use game trees to solve games, i.e., to find a sequence of moves that one of the players can follow to guarantee their win. Thus, if we have access to the complete game tree, we can calculate $\eta(A,s)$ by checking all terminal states that are reachable from $s$ by a path of strictly increasing depth.

**Q6.** The function $\eta(A,s)$ considers how many reachable terminal states are wins or losses for player A. Why can $\eta(A,s) > 0$ still fail to show that A can force a win against a rational opponent? Illustrate your answer with a small game-tree example or a concrete game situation.

[1] Note that there may be more than one node with the same state.

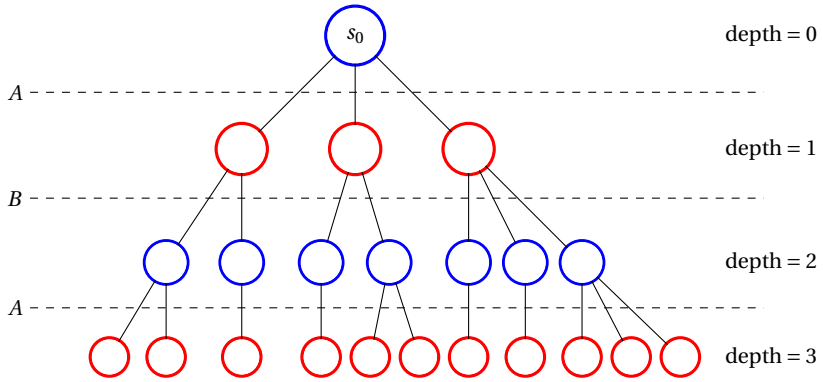



Figure 2.1: Each node corresponds to a game state, The blue nodes have even depth, the red nodes have odd depth, and there is an edge from a blue node to a red node (with corresponding states $s_b, s_r$, respectively) if in state $s_b$, player $A$ can perform a move that will change the state into $s_r$ and there is an edge from a red node to a blue node (with corresponding states $s'_r, s'_b$, respectively) if in state $s'_r$, player $B$ can perform a move that transforms state into $s'_b$.

## 2.3 Minimax algorithm

Although $\eta$ uses information about possible continuations, it ignores which player controls each choice and gives equal weight to all reachable terminal states. It therefore does not represent the behavior of a rational opponent (see Figure 2.2 for an example).
In the specific case of $\eta$, consider for instance $\eta(A, s) = 10$ for a given state $s$. This just means that there are 10 more reachable terminal states that result in a victory for player $A$ than for player $B$, however it tells nothing of the distribution of such states, and how much $B$ can do to counteract those moves.

The way around this problem is assuming that the opponent is also aware that it has to optimize its chances of winning, and therefore when calculating a heuristic function that requires traversing the tree, we should only consider the transitions that maximize the heuristic function for player $B$ (therefore minimizing the heuristic for player $A$). This can be done by employing the *minimax algorithm*, whose pseudocode is shown in Algorithm 1.

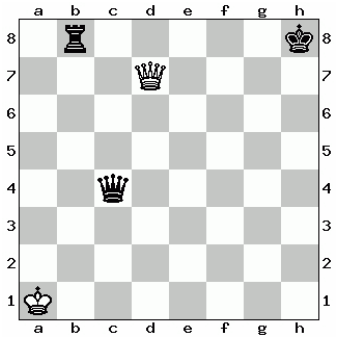


Figure 2.2: Consider this chessboard setup, with black to play. While playing white an algorithm that only uses a heuristic function such as $\eta$ may treat the black queen moving horizontally to $a4$ in the same way as moving diagonally to $a6$ while any player would be able to see that moving the queen to $a4$ would result in the loss of the queen, and most likely the loss of any chance of mate by the black player, whereas moving the queen diagonally would result in mate in two turns.

Algorithm 1: Pseudocode for the minimax algorithm

```
1  int minimax (state, player)
2      // state: the current state being analyzed
3      // player: the player whose turn it is
4      // returns the minimax value from the perspective of player A (MAX)
5
6      if μ(player, state) = ∅
7          // terminal state; γ is the utility from A's perspective
8          return γ(A, state)
9
10     else
11         if player = A
12             bestPossible = −∞
13             for each child in μ(A, state)
14                 v = minimax(child, B)
15                 bestPossible = max(bestPossible, v)
16             return bestPossible
17
18         else // player = B
19             bestPossible = +∞
20             for each child in μ(B, state)
21                 v = minimax(child, A)
22                 bestPossible = min(bestPossible, v)
23             return bestPossible
```

Note that Algorithm 1 requires one to evaluate the tree until it reaches a terminal state. However, given any evaluation function which can assign a heuristic for the game, we can truncate the search at a given depth by including the number of levels left to analyze as an argument and decreasing it in each step.

## 2.4 Alpha-Beta pruning

Now, Algorithm 1 requires one to search the complete game tree (at least until a given depth). However, this may not be necessary. The idea behind the *alpha-beta pruning* algorithm is that in some situations, parts of the tree may be ignored if we know a priori that no better result may be achieved.

To understand how this is possible, consider that player $A$ is traversing the tree in order to find the next best move using the minimax algorithm. Instead of just keeping track of the best node found so far, the player can also keep track of the best heuristic value computed so far $\alpha$ as well as the worst such value $\beta$ (which yields the best result for $B$). At each transition in the tree $\alpha$ should be updated, if it is player $A$'s turn, and otherwise $\beta$ should be updated. Finally, the remainder of a branch should be disregarded whenever $\alpha$ is greater than or equal to $\beta$, since that indicates the presence of a non-desirable state.

Formally, if $\alpha$ gets updated in player's A turn and $\alpha \geq \beta$, then we already know that

$$\max(\alpha, \min(\beta, v)) = \alpha,$$

regardless of which is the value $v$, and since $\alpha$ is already the best that player $A$ can do, it is worthless to explore any other state. Similarly, if $\beta$ gets updated in player's B turn and $\alpha \geq \beta$, then we already know that

$$\min(\beta, \max(\alpha, v)) = \beta,$$

regardless of which is the value of $v$.

## 2.5 Further optimizations

Until now, we saw that the *minimax* algorithm allows us to search through the game tree for the optimal strategy (assuming a perfect opponent) based on a given heuristic. Then, we observed how *alpha-beta pruning* allows us to reduce the game tree search by skipping entire branches that cannot affect the minimax result.

However, alpha-beta pruning is not the only addition we can perform to the minimax algorithm in order to make our game tree search more efficient. It is now up to **you** to check on other techniques such as *iterative deepening search*, *move ordering*, *repeated-state detection or transposition tables*, *quiescence search*, *forward pruning*, and more (see [1, Chapter 5]) in order to incorporate them in your algorithm implementation for this assignment.

# 3 THE ASSIGNMENT

In this assignment, you will implement an agent that selects the next move for the green player in the KTH fishing derby game. Your agent will be tested in scenarios of increasing difficulty against the red opponent. Its performance will depend on the correctness and efficiency of its game-tree search and on the quality of its evaluation function.
The program and Kattis submission structure are described here:

https://kth.kattis.com/courses/DD2380/ht26/assignments/mitsx2/problems/fishingderby

Your agent must return an action in less than 75 milliseconds on each time step. Exceeding this limit on Kattis will stop the evaluation with a run-time error.

## 3.1 HOW THE ASSIGNMENT GRADE IS DETERMINED

The assignment is assessed using both the submitted implementation and the individual Assignment Learning Journal.

- The **Kattis result** determines the highest grade range for which the implementation is eligible.
- The **Assignment Learning Journal** provides evidence of the student's understanding, design decisions, evaluation of the solution, and individual learning.
- A TA reviews the journal and code and assigns the **final assignment grade**. If evidence is insufficient, the grade may be lower than the maximum permitted by the Kattis result.
- The later **individual oral examination** validates that the student understands the submitted assignments and may cap the final LAB2 grade.

The assignment grade is the highest grade for which the implementation and journal together provide sufficient evidence, subject to the Kattis eligibility thresholds below.

READING YOUR KATTIS SCORE. In Kattis, the number of completed test cases is not the same as your assignment score. For example, a submission displayed as `Accepted (20)` with 25/25 test cases has earned 20 points out of 25, not 25 points. The number in parentheses after `Accepted` is the score used for the grade thresholds below. Always check this score before recording it in your Assignment Learning Journal.

### 3.2 GRADE LEVELS E AND D

For eligibility in the E–D range, you must implement minimax with a suitable evaluation function and a search depth of at least 2. Because the complete game tree cannot normally be searched within the available time, the search must use an appropriate cut-off.
**Kattis requirement:** at least 10 points out of 25 and a search depth of at least 2. In this assignment, the root node has depth 0, and each transition from a node to one of its children increases the depth by 1.
Meeting this threshold makes the implementation eligible for E or D. The distinction between E and D is based on the evidence of understanding, decisions, and learning provided in the journal.

### 3.3 GRADE LEVEL C

For eligibility for C, your implementation must combine a correct minimax search and evaluation function with techniques that allow it to search effectively under the available time constraints. Relevant techniques may include alpha-beta pruning, iterative deepening, or move ordering.
**Kattis requirement:** at least 15 points out of 25.
Meeting this threshold makes the implementation eligible for C. The journal must confirm C-level understanding of how the search, evaluation function, cut-off, and efficiency techniques interact.

### 3.4 GRADE LEVELS B AND A

For eligibility in the B–A range, your implementation must be capable of handling the more difficult scenarios. This normally requires advanced search techniques in addition to a basic minimax implementation, such as effective move ordering, repeated-state handling, iterative deepening, or other justified optimizations.
When caching search results or detecting repeated states, two nodes may only be treated as equivalent if all information relevant to their future search is equivalent, including the player to move and the relevant position in the observation sequence.
Simply changing constants, increasing the nominal depth, or repeatedly adjusting the evaluation function until the required Kattis score is reached is not by itself sufficient evidence for B or A.
**Kattis requirement:** at least 20 points out of 25.
Meeting this threshold makes the implementation eligible for B or A. Grade B requires a justified advanced technique supported by evidence and an understanding of its limitations. Grade A additionally requires a critical, evidence-based comparison with a reasonable alternative, explaining the trade-offs and justifying the chosen approach. Grade A also requires on-time submission.

# 4 Assignment Learning Journal

The Assignment Learning Journal is an individual, concise account of what you implemented, why you made important decisions, and what you learned. It is not a diary or a full technical report. Connect your explanations to your submitted code and observations.

> The Kattis result determines the highest eligible grade. The journal provides evidence of your individual understanding within that range. A TA reviews the journal and code and assigns a final grade. The journal and code may also be used to prepare questions for the individual oral examination.

## 4.1 Format and submission

Use the headings J0–J6 and submit the journal as instructed in Canvas. Aim for 700–1000 words; the maximum is 1200 words, excluding the question prompts and J6. Short paragraphs and bullet points are welcome. Do not paste source code or screenshots; refer to files, classes, or functions by name.
If you worked with a partner, the implementation may be shared, but the journal must be written individually. Shared facts may overlap; your explanations, contribution, and learning must be your own. Assessment concerns technical understanding, not writing style.

J0 – Submission record. Give your name and KTH email, your partner's name (or "none"), your target grade, and exactly one Kattis submission ID with its actual score (out of 25). This must be the submission you want assessed. Refer to other submissions in J3 or J4 if relevant, not here.

J1 – Solution overview (about 100–150 words). Explain your solution's central idea and how its main components work together, from the input or problem state to the output or decision. Name the important files, classes, or functions. Distinguish provided skeleton code from what you implemented or substantially changed.

J2 – Assignment questions. Answer every numbered question in the assignment, normally in about 40–80 words each. Use a small example, formula, trace, or code reference when helpful. Explain in **your own words** rather than only giving a definition.

J3 – Decisions and evidence. Describe two important decisions concerning, for example, the algorithm, model, representation, evaluation, parameters, efficiency, or testing. For each, give the goal, alternatives considered, choice and reason, code location, evidence, and a trade-off or limitation. Evidence may be a controlled comparison, local test, trace, measurement, Kattis result, or failure case. At least one decision must include a revised approach,

unexpected behavior, important boundary case, remaining limitation, or meaningful alternative you evaluated.

J4 – ADVANCED ANALYSIS (REQUIRED WHEN AIMING FOR B OR A). Build on one of the decisions described in J3. Analyze the advanced technique or design choice in more depth: explain how it works, why it should help, evidence of its effect, and a limitation or trade-off. Describe a reasonable alternative. For grade A, critically compare the alternatives using concrete evidence and justify your choice under the assignment's constraints.

J5 – INDIVIDUAL LEARNING AND CONTRIBUTION (ABOUT 100–150 WORDS). State what you personally contributed. Describe the most important thing you understand now that you did not understand before, the hardest issue you resolved, and one limitation or open question that remains.

J6 – USE OF GENERATIVE AI. State whether you used any generative AI tools while developing the code or writing the journal. If you did not, write "No generative AI tools were used." If you did, name the tools and briefly describe what you used them for, which parts of your submission they influenced, and how you checked, corrected, or rejected their suggestions. Do not paste complete prompts or conversations. You remain responsible for understanding and verifying everything you submit.

## REFERENCES

[1] S. Russell and P. Norvig, *Artificial intelligence: a modern approach*. Prentice Hall, 3 ed., 2010.